

Malleable JavaScript for Assurance-Oriented Systems

*How Interpreter and Compiler Control Could Evolve Tiny-IDP
into a Reusable, Verifiable Plugin Platform*

A source-grounded architecture report and research roadmap

CORE RECOMMENDATION

Build a JavaScript-compatible effect-aware front end and a small, canonical semantic IR; add narrow Goja VM instrumentation only for guarantees unavailable above the VM; do not persist VM frames and do not begin with a new full JavaScript engine.

SOURCE SNAPSHOT

Tiny-IDP branch task/prod-tiny-idp at d164ae59408bdd8bc21516274b446339b1761b1e

Goja at af2ceb9156d7feaff65273b8bfde778077fb4b7e

Research current through 2026-07-20

Contents

The entries below link to the principal sections. The report uses stable source and semantic terminology throughout; the glossary is in Appendix E.

Scope, terminology, and source snapshot

Abstract

Executive synthesis

- The architecture to build
- What changes for Tiny-IDP
- What changes for script authors
- What requires VM modification
- What should be verified
- Recommendation in one sentence

Part I - The baseline and the design question

1. Tiny-IDP already implements a language runtime, not merely an embedding
2. What owning more of the interpreter changes
3. Five levels of control

Part II - Improvements available above the VM

4. Define an assurance-oriented JavaScript profile
5. Infer capabilities and effects
6. Generate deterministic callback identities
7. Harden the JavaScript world

Part III - Compiler transformations and new programming models

8. Durable await as compiled syntax sugar
9. Live-variable analysis defines continuation carry
10. Structured concurrency for capabilities and Promises
11. Secret, evidence, and provenance analysis
12. Compile effects instead of interpreting arbitrary result objects
13. Extract state machines and properties automatically

Part IV - What direct VM access would enable

14. Deterministic instruction fuel
15. Allocation and retained-heap accounting
16. Job-queue control and deterministic Promise semantics
17. Internal provenance and unforgeable value classes
18. Semantic tracing, debugging, and explainability
19. Runtime snapshots and reset: useful, but not durable control flow
20. Why VM-frame persistence is the wrong continuation model

Part V - A project-independent semantic intermediate representation

21. Why an MIR is the strategic center
22. Core MIR concepts
23. An illustrative MIR
24. Reference interpreter and optimized backends
25. Proof-carrying activation artifacts
26. Formal verification strategy

Part VI - Making the compiler malleable across projects

27. Dialects, worlds, passes, and backends
28. Declarative dialect definitions
29. Extension trust tiers
30. Tiny-IDP as a dialect and world pack
31. Other project families
32. Multi-frontend architecture

Part VII - The plugin-writer experience

33. From manual contracts to inferred contracts
34. Diagnostics as the primary usability feature
35. IDE and review tooling
36. Testing and simulation

37. Packaging, linking, and reusable libraries

38. Semantic hot reload and migration

Part VIII - Evaluation, risks, and roadmap

39. Option matrix

40. Principal risks and mitigations

41. Staged implementation roadmap

42. Concrete first experiments

43. Final recommendations

Appendix A - Proposed core interfaces

A.1 World manifest

A.2 Dialect operation

A.3 MIR function

A.4 Suspension terminator

Appendix B - Static check catalogue

Appendix C - Example activation certificate

Appendix D - Comparison with adjacent systems

Appendix E - Glossary

References

Scope, terminology, and source snapshot Contents

This report studies how deeper control over the JavaScript implementation could change the architecture developed in `go-go-golems/tiny-idp` under ticket TINYIDP-GOJA-001. It considers five progressively more invasive options:

1. use only Goja's public embedding API;
2. add a JavaScript front end and source-to-source or AST transformation layer around Goja;
3. maintain a narrowly instrumented Goja fork;
4. define and interpret a project-independent intermediate representation while retaining JavaScript as a source language;
5. implement a new JavaScript interpreter or compiler.

The Tiny-IDP source snapshot is branch `task/prod-tiny-idp` at commit [d164ae59408bdd8bc21516274b446339b1761b1e](#), dated 2026-07-20. That branch pins Goja commit [af2ceb9156d7feaff65273b8bfde778077fb4b7e](#), dated 2025-11-03. Current public documentation and related language/runtime systems were also reviewed as of 2026-07-20.

The report distinguishes three categories of statement:

- **Current implementation** describes code present in the pinned Tiny-IDP branch.
- **Near-term proposal** describes changes that can plausibly be built above Goja's public parser/compiler/runtime APIs.
- **Research direction** describes a larger language-platform effort whose cost, compatibility burden, and proof obligations are materially higher.

The term *verified* is used narrowly. A compiler can have verified passes or a verifier with a soundness proof without making all JavaScript programs, all host capabilities, or the whole identity provider formally verified. The term *novel* refers to the engineering synthesis proposed here, not to academic priority.

Abstract

Contents

Tiny-IDP already uses JavaScript in an unusually disciplined way. Source is evaluated in an isolated definition-time runtime; callback closures remain VM-local; a pure-Go program contract records callback IDs, schemas, capabilities, outcomes, effects, and budgets; request-time values cross the boundary through JSON; capabilities are invocation-scoped; browser waits become explicit durable continuations; and Go retains authority over protocol validation, secrets, browser security, persistence, challenges, commits, and artifact issuance.[1][2]

Deeper access to the JavaScript compiler and interpreter could convert many of these host-enforced conventions into language-level guarantees. A compiler could infer capability and effect requirements, reject accidental ambient authority, generate stable callback identities, lower browser-spanning `await` into typed state machines, compute the exact live variables that may cross a continuation boundary, enforce structured concurrency, track secret and evidence provenance, produce replayable semantic traces, and emit a proof-carrying activation artifact. A selectively modified VM could add deterministic instruction fuel, allocation accounting, job-queue control, per-value provenance, and precise interruption and tracing hooks.

The central recommendation is not to implement a complete JavaScript engine first. Full ECMAScript implementation brings a continuing specification, conformance, parser, regular-expression, Unicode, module, garbage-collection, debugger, performance, and security burden.[3][4] The better architecture is a **malleable JavaScript substrate** with four layers:

```

TEXT
standard JavaScript surface syntax
    |
    v
profile checker + typed/effectful front end
    |
    v
small project-independent semantic IR
    |
    +--> Goja compute backend
    +--> native reference interpreter
    +--> workflow/state-machine backend
    +--> static analysis and model checking
    +--> documentation, tests, and IDE metadata

```

Projects extend the substrate through versioned **dialects** and **worlds**. A dialect defines closed types, operations, effects, verifiers, lowerings, documentation, and test generators. A world selects the exact dialect versions and capability imports available to one project. Tiny-IDP becomes one assurance-sensitive world rather than the hard-coded owner of the language platform. This borrows useful structure from MLIR dialects, WIT worlds, object-capability systems, CEL's checked serialized AST, and formalized policy languages, while preserving a familiar JavaScript authoring experience.[5][6][7][8]

The most important negative conclusion is equally clear: durable continuations should not be implemented by serializing Goja's suspended VM frames. Goja's internal suspended execution state contains runtime-local values, lexical environments, stacks, references, and program counters. Persisting that state would bind durable records to engine internals, complicate secret erasure, heap migration, replay validation, garbage collection, and hot reload. Durable `await` should be syntax sugar compiled by continuation-passing transformation and defunctionalization into the explicit, versioned native continuation model Tiny-IDP already implements.

Executive synthesis Contents

The architecture to build

The proposed system, called **Malleable JS** in this report, is a compiler and interpreter substrate rather than a new general-purpose JavaScript runtime. It should preserve standard JavaScript where possible, reject or transform constructs that violate a selected assurance profile, and lower accepted programs to a stable semantic intermediate representation (MIR).

The MIR is the long-lived contract. It contains:

- closed value and schema types;
- functions and basic blocks with stable identities;
- declared or inferred capability imports;
- an effect row for each function;
- resource and region types for secrets, evidence, transactions, and invocation-scoped handles;
- explicit suspension points and continuation carry schemas;
- closed outcomes and typed effect plans;
- source provenance and diagnostic locations;
- resource budgets and verification results.

The JavaScript source and Goja bytecode are replaceable front-end/backend artifacts. Continuations, activation identity, model checking, upgrade compatibility, and verification operate on the MIR.

What changes for Tiny-IDP

Today Tiny-IDP constructs `LambdaSpec` values at definition time and validates actual behavior at invocation time. With compiler ownership, a large part of `LambdaSpec` can be inferred and checked before activation:

TEXT	
current declaration	compiler-derived contract
capabilities: ["community.lookup"]	inferred from reachable capability calls
effects: ["create_identity"]	inferred from returned effect constructors
outcomes: ["present", "commit"]	inferred as a closed union of return paths
input/output schema IDs	resolved and type-checked at property access
callback ID	generated from module/export/lexical identity
continuation carry schema	derived from liveness at durable suspension
call/output budget	declared policy plus compiler estimates
source location	precise source map and IR provenance

The host remains authoritative. Inference does not grant power. The world manifest still states the maximum capabilities and effects a project permits. The compiler proves that the program requests a subset of that maximum; the runtime supplies only the approved imports.

What changes for script authors

A plugin author can write code resembling ordinary asynchronous JavaScript:

```
JAVASCRIPT
export async function signup(ctx) {
  const invite = await ctx.invites.inspect(ctx.input.inviteCode);

  const form = await durable.form(SignupForm, {
    email: invite.email,
  });

  const proof = await durable.emailCode({
    address: form.email,
  });

  return effects.commit([
    identity.create({ email: form.email, verifiedBy: proof }),
    invite.consume(invite),
    session.establish(),
  ]);
}
```

```
}

```

The compiler rejects this program if `invite` contains non-serializable or secret state live across `durable`.form, if the world does not import `invites.inspect`, if a detached Promise can survive the invocation, if the commit sequence is illegal, or if an outcome is not covered. It lowers the function to named handlers and continuation records compatible with the native Tiny-IDP executor.

This is more usable because the author writes local control flow instead of manually naming every resume handler and copying every carry field. It is more robust because the compiler, not the author, computes continuation state. It is more analyzable because the result is an explicit state machine. It is more maintainable because source identities, migrations, and effect contracts are generated rather than duplicated.

What requires VM modification

Most semantic improvements do not require a Goja fork. Goja exposes parsing and `CompileAST`, and compiled programs are runtime-independent. A front end can therefore parse, analyze, transform, and then hand accepted code back to Goja.[9]

A fork becomes justified for guarantees that cannot be observed reliably at the host boundary:

- deterministic instruction-level fuel;
- complete allocation and retained-heap accounting;
- controlled Promise job scheduling and microtask quiescence;
- precise semantic trace probes at calls, branches, throws, allocations, and awaits;
- host-internal provenance tags that JavaScript cannot forge or erase;
- stronger runtime reset or isolate lifecycle hooks;
- a stable compiler IR export, if source-level transformation proves too brittle.

Even then, the fork should be a small, generic instrumentation layer suitable for upstreaming. It should not contain Tiny-IDP-specific opcodes or protocol logic.

What should be verified

The feasible verification target is the small semantic core, not full ECMAScript:

1. define a formal small-step semantics for MIR values, control flow, effects, regions, and suspension;
2. prove the MIR verifier's principal safety properties;
3. prove selected lowering passes preserve observable effect traces for the accepted JavaScript profile;
4. differentially execute MIR programs in a simple reference interpreter and the Goja backend;
5. generate conformance and property tests from dialect definitions;
6. emit an activation certificate whose hashes and verifier results are checked by the production runtime.

Cedar's verified validator and differential comparison with the production implementation provide a useful model: prove a tractable semantic component and continuously compare it to the optimized implementation.[7] JSCert and KJS demonstrate that mechanized JavaScript semantics are possible, but they also illustrate the scale and versioning cost of targeting the full language.[10][11]

Recommendation in one sentence

Build a JavaScript-compatible, effect-aware compiler front end and a small extensible MIR now; add narrowly scoped Goja VM instrumentation only when a concrete guarantee cannot be obtained above the VM; do not serialize VM frames and do not begin with a new full JavaScript engine.

Part I - The baseline and the design question Contents

1. Tiny-IDP already implements a language runtime, not merely an embedding

The existing system has at least four semantic stages.

1.1 Definition-time evaluation

Trusted JavaScript runs in an isolated owned Goja runtime. The only native module is `require("tinyidp");`; ambient module loading is disabled. The module collector records lambda specifications and VM-local callable values. The exported result is serialized as a pure-Go `idpprogram.Program.[1][2]`

The separation is exact:

TEXT	
VM-local, executable	VM-independent, durable
-----	-----
goja.Callable	callback ID
opaque branded JS object	schema/effect/action identifier
mutable JS object graph	canonical JSON and Go structs
Promise	closed Outcome value
host function closure	capability requirement

1.2 Activation and reproducible materialization

The same compiled Goja program is loaded into independent runtime images. Tiny-IDP compares canonical exported program data, collector program data, program fingerprints, callback-registry fingerprints, schema fingerprints, and the set of callback IDs. A runtime whose definition-time behavior differs from the artifact is rejected.[2]

This is a linking discipline. The serializable program says `signup.submitted`; each worker must reproduce exactly one local closure under that name.

1.3 Request-time invocation

A worker is leased exclusively. Input is validated, decoded through `JSON.parse` into ordinary JavaScript values, assembled with only the selected lambda's capability bindings, secret handles, evidence, presentation builders, challenge builders, and commit builders, then transitively frozen. Synchronous values or bounded Promises are converted back to JSON and checked against a closed outcome contract.[12]

Capability operations are versioned, byte-bounded, call-bounded, invocation-scoped, and settled through the runtime owner. A retained function loses authority once the invocation closes.[13]

1.4 Native workflow interpretation

JavaScript outcomes are proposals, not final protocol effects. Native Go code validates and interprets presentations, challenges, continuations, commits, completions, denials, skips, and errors. Browser waits persist pure-Go continuation state containing a handler ID, generation identity, typed carry, native references, revision, expiry, and replay-sensitive bindings. No Goja value, closure, Promise, or goroutine state is durable.[14]

The current design is therefore already a staged, capability-oriented interpreter with an explicit intermediate contract. Compiler access does not create this architecture from nothing. It allows the architecture to move selected invariants earlier, make them compositional, and reduce authorial duplication.

2. What owning more of the interpreter changes

An embedding API exposes values and calls. A compiler exposes program structure. A VM exposes execution structure. Those three levels support different guarantees.

2.1 At the embedding boundary

The host can validate concrete inputs, outputs, capabilities, timeouts, and worker lifecycle. It can observe that a script called `community.lookup`, but only after the call occurs. It can reject an undeclared effect after the script returns it. It

generally cannot determine all reachable calls, all possible outcomes, all variables live across a source-level suspension, or whether a Promise was intentionally detached.

2.2 At the compiler boundary

The system can reason over syntax, lexical scope, control flow, and data flow before execution. It can produce stable semantic identities, infer effects, transform control flow, attach source provenance, and extract a state graph. It can reject a program because of what it *could* do, not merely because of what one invocation did.

2.3 At the VM boundary

The system can account for executed instructions, allocations, job-queue operations, runtime-local identities, and interruption points. It can observe behavior that source analysis approximates poorly, including metaprogramming, polymorphic dispatch, dynamic property access, and library code.

2.4 With a separate semantic IR

The system can give project invariants a representation independent of JavaScript syntax and Goja internals. The same program can be executed, modeled, documented, migrated, or compiled through multiple backends. Formal proofs can target a small language designed for the relevant properties.

2.5 With a complete new JavaScript implementation

Every semantic choice is controllable, but every ECMAScript obligation becomes local. The project owns language evolution, Test262 conformance, object model corner cases, numerical behavior, regular expressions, Unicode, modules, source maps, garbage collection, debugging, profiling, performance, and a permanent vulnerability surface. This is a strategic platform commitment, not an implementation shortcut.

3. Five levels of control

Level	Description	Main benefits	Main limitations
A	Public Goja embedding APIs	Low maintenance; current architecture; host-defined capabilities and lifecycle	Mostly dynamic checks; weak resource accounting; limited source reasoning
B	Goja parser/AST plus compiler front end	Static profiles, inference, transforms, state-machine extraction, precise diagnostics	Goja AST is documented as work in progress; runtime behavior still partly opaque
C	Narrow Goja fork	Fuel, allocation quotas, job control, provenance, trace hooks	Upstream divergence and VM maintenance
D	Own restricted MIR and reference interpreter	Stable semantics, multiple backends, formal verification, project dialects	New compiler and tooling platform to design and maintain
E	Own complete JavaScript engine	Maximum control and experimentation	Very high conformance, performance, security, and ecosystem cost

The recommended trajectory is **B + D**, followed by selective **C**. Level E should be considered only after the organization explicitly decides to maintain a language runtime as a product in its own right.

Part II - Improvements available above the VM Contents

4. Define an assurance-oriented JavaScript profile

The first compiler feature should be a profile checker, not new syntax. Standard JavaScript source is parsed, then accepted, warned, or rejected under a named world profile.

A Tiny-IDP profile might prohibit or restrict:

- direct and indirect `eval` and the `Function` constructor;
- dynamic import and unapproved static imports;
- writes to `globalThis` or imported module namespaces;
- mutation of standard prototypes and constructors;
- dynamic property access on capability namespaces;
- `with`, sloppy mode, and legacy syntax with surprising scoping;
- unbounded recursion or loops in handlers that require a static bound;
- getters, setters, proxies, or `Symbol` use at security boundaries;
- detached Promises and unhandled rejections;
- implicit time, randomness, locale, environment, filesystem, process, network, or module-loader access;
- top-level mutable state for stateless or deterministic worker profiles.

Hardened JavaScript systems use a related strategy: remove ambient I/O, freeze primordials, disable nondeterministic time and randomness, and provide explicit endowments to compartments.[5] Starlark demonstrates the value of a deliberately smaller, deterministic, hermetic language for configuration and build logic.[15] Tiny-IDP need not copy either language, but it should define exactly which JavaScript it accepts rather than relying on the absence of a few globals.

4.1 Profiles should be named and versioned

A source artifact should declare or be compiled under a world such as:

```
TEXT
world tinyidp:policy@2.0 {
  language-profile: assurance-js@1.3
  imports:
    tinyidp:community.lookup@1
    tinyidp:challenge.email@2
    tinyidp:identity.commit@1
  limits:
    instructions: 200000
    allocation-bytes: 1048576
    capability-calls: 8
    durable-carry-bytes: 8192
}
```

The profile version becomes part of activation identity. A compiler upgrade cannot silently reinterpret an existing program under changed restrictions.

4.2 Profile checking is not sandboxing

Static rejection reduces the accepted language and improves analysis. It does not make arbitrary hostile code safe in-process. Prototype semantics, engine bugs, denial of service, native capability bugs, and side channels remain. Untrusted third-party plugins still need process, operating-system, or WebAssembly isolation in addition to language controls.

5. Infer capabilities and effects

The current `LambdaSpec` explicitly declares required capabilities and allowed effects.[16] That is sound at runtime but duplicates facts visible in source. A compiler can construct a call/effect graph.

For statically named capability paths:

```
JAVASCRIPT
const member = await ctx.cap.community.lookup({ email });
```

it can infer the effect atom:

```
TEXT
cap<community.lookup@1>
```

For builders:

```
JAVASCRIPT
return ctx.commit.signup({ ... });
```

it can infer a closed outcome and effect row:

```
TEXT
outcome<commit> + effect<signup.commit>
```

The inferred set must be a subset of the world grant. This reverses a common mistake: declaration is not authority. The world grants the maximum; source analysis computes what this artifact actually imports.

5.1 Effect rows for JavaScript handlers

A function type can be represented conceptually as:

```
TEXT
signupSubmitted : SubmittedInput
-> <cap.community.lookup,
   challenge.email,
   present.signup,
   deny>
   Outcome<SignupResult>
```

This resembles typed effect rows used in languages such as Koka, where exceptions, asynchronous operations, generators, and other control abstractions can be expressed and composed through effects and handlers.[17]

The compiler need not expose advanced type theory to script authors. It can surface concise diagnostics:

```
TEXT
TIDP-E204: handler signup.submitted calls community.lookup,
but world tinyidp:signup@4 does not import that capability.

41 | const member = await ctx.cap.community.lookup({ email });
    |                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

5.2 Dynamic dispatch policy

JavaScript permits computed property access:

```
JAVASCRIPT
ctx.cap[namespace][operation](input)
```

A high-assurance profile should reject this on capability roots unless the index is a compile-time finite union. Otherwise the compiler must conservatively grant an entire namespace, defeating least authority.

5.3 Transitive inference across libraries

Reusable libraries should publish summaries containing their types and effects. Linking a library should compose those summaries without re-trusting handwritten metadata. When source is available, the compiler verifies the summary. When only a signed artifact is available, the activation certificate binds the summary to the semantic IR hash.

6. Generate deterministic callback identities

Tiny-IDP currently asks the author to supply stable callback IDs and verifies that each runtime recreates the same registry.[2] Compiler ownership permits a more ergonomic rule:

```
TEXT
callback-id = package-id / module-path / export-or-lexical-path / explicit-generation
```

For example:

JAVASCRIPT

```
export const signup = workflow({
  submitted: async ctx => { ... }
});
```

could receive:

TEXT

```
community-idp/signup.js::signup.submitted
```

An explicit annotation remains available for migrations:

JAVASCRIPT

```
/** @stableId community.signup.submitted.v2 */
submitted: async ctx => { ... }
```

6.1 Identity must follow semantics, not line numbers

Line/column hashes are unstable under formatting and comments. Function-body hashes are unstable under innocuous rewrites and cannot preserve identity across compatible migrations. The correct scheme combines a declared package/module namespace with a lexical declaration path and an optional explicit stable ID.

6.2 Detect identity collisions and accidental renames

The compiler can compare a prior activation manifest and report:

TEXT

```
TIDP-M101: durable handler identity removed:
  community.signup.after_email@1
```

Active continuations reference this identity. Add @migrateFrom, retain a compatibility handler, or explicitly revoke the generation.

This turns hot-reload compatibility into a compiler-visible API change rather than an operational surprise.

7. Harden the JavaScript world

Tiny-IDP already disables ambient module loading and deep-freezes invocation context values.[2][12] Compiler and runtime control can make the whole language world more explicit.

7.1 Frozen primordials

A plugin should not be able to modify `Array.prototype.map`, replace `Object.freeze`, or use shared built-ins as a covert communication channel between invocations. A hardened world can initialize intrinsics once, remove disallowed properties, and transitively freeze them before plugin code runs. This follows the object-capability rationale used by Hardened JavaScript compartments.[5]

7.2 Separate compartments or realms

Goja runtimes are already isolated from one another, but multiple packages within one runtime may still share globals and intrinsics. A project-independent platform should distinguish:

- **realm**: a full set of intrinsics and global object;
- **compartment**: a module/global namespace with explicit imports and hardened shared intrinsics;
- **invocation region**: one temporary capability/evidence/secret grant.

The TC39 ShadowRealm proposal illustrates the value of distinct global environments and intrinsics, though its standardization status and API should not be treated as a production dependency.[18]

7.3 Deterministic time and randomness

Goja exposes host-settable time and random sources.[9] Production worlds should make `Date.now`, zero-argument `Date`, and `Math.random` unavailable or route them to explicit effects. Tests can supply virtual clocks and seeded random streams. Durable workflows must journal any nondeterministic result that influences future control flow.

7.4 Module state policy

Worker reuse means top-level mutable module state can survive between invocations. That may be intentional for caches, but it complicates determinism, tenant isolation, and testing. Worlds should choose one of three policies:

1. **pure module** - top-level state freezes after activation;
2. **ephemeral worker state** - mutation allowed but not observable as a semantic dependency; workers may be replaced at any time;
3. **declared state resource** - state is an explicit host resource with versioning, quotas, and reset semantics.

The default for identity policy should be pure module state.

Part III - Compiler transformations and new programming models Contents

8. Durable `await` as compiled syntax sugar

The Tiny-IDP design explicitly notes that a future source transform could offer browser-spanning `await`, provided it compiles to explicit continuations.[1] This is the highest-value compiler feature.

8.1 Two kinds of `await`

The compiler must distinguish:

- **ephemeral `await`**: waits for a bounded capability during one request and one leased runtime;
- **durable `await`**: ends the current request, persists a continuation, and resumes in a fresh invocation after an external event.

A type/effect distinction makes the boundary visible:

```
TEXT
Promise<T>           - ephemeral, invocation-scoped
Durable<T, ResumeTag> - persistence boundary
```

Only a durable.* operation can produce the second form.

8.2 Source transformation

Consider:

```
JAVASCRIPT
export async function signup(ctx) {
  const invite = await ctx.cap.invites.inspect(ctx.input.code);
  const form = await durable.form(SignupForm, { email: invite.email });
  const proof = await durable.emailCode({ address: form.email });
  return commit.signup({ email: form.email, proof });
}
```

The compiler can perform continuation-passing transformation and defunctionalization:

```
TEXT
handler signup.entry(input):
  invite = effect invites.inspect(input.code)
  return present SignupForm
    resume signup.after_form
    carry { inviteRef: invite.ref, suggestedEmail: invite.email }

handler signup.after_form(event, carry):
  form = decode SignupForm event
  return challenge email_code(form.email)
    resume signup.after_email
    carry { email: form.email }

handler signup.after_email(evidence, carry):
  return commit signup(
    email = carry.email,
    proof = evidence.email_proof
  )
```

The output is compatible with the existing `BrowserContinuation` and `WorkflowContinuation` structures.[14]

8.3 Why this is better than serializing a `Promise`

The persisted record contains only stable labels and schema-checked data. It does not depend on:

- the Goja heap layout;
- closure representation;
- lexical-environment pointers;
- the `Promise` job queue;
- goroutine identity;
- a specific worker;

- a specific VM version;
- live secrets or native capability functions.

It remains inspectable, migratable, revocable, and replay-protected by native code.

9. Live-variable analysis defines continuation carry

At each durable suspension, the compiler computes which values may be used after resumption. This is a standard liveness problem with unusually important security consequences.

For:

```
JAVASCRIPT
const password = ctx.secret.password;
const invite = await ctx.cap.invites.inspect(code);
const form = await durable.form(Form, { email: invite.email });
return commit({ password, form });
```

`password` is live across the durable form boundary. Its type is an invocation-scoped secret resource. The compiler must reject the program:

```
TEXT
TIDP-S312: invocation-scoped secret `password` is live across a durable suspension.
Secret values and handles cannot be persisted or recreated after browser resume.

12 | const form = await durable.form(Form, { email: invite.email });
    | ~~~~~
13 | return commit({ password, form });
    | ~~~~~ used after suspension
```

9.1 Continuation-safe types

Every type should have a persistence classification:

Class	Examples	May cross durable suspension?
Public value	bounded string, boolean, enum, public record	Yes, under schema and size bound
Sensitive durable reference	invitation ID, challenge ID, encrypted blob reference	Yes, as opaque native reference
Evidence	verified email evidence, WebAuthn assertion result	Only by host-issued evidence reference or typed projection
Secret	password bytes, raw code, private key	No
Invocation resource	capability function, transaction, HTTP request, logger span	No
VM-local value	function, Promise, Proxy, Symbol, object with cycles	No
Recomputable pure value	normalized display label	Prefer recomputation; may be carried if bounded

9.2 Carry minimization

Even public values should not be persisted automatically without policy. The compiler can minimize carry to the exact live fields, then enforce a world-level size and sensitivity budget. This reduces data retention and migration burden.

9.3 Explicit escape hatches

Some values may be intentionally converted to durable references:

```
JAVASCRIPT
const reservation = await durable.storeRef(invite);
```

That operation is an effect with a native implementation, audit trail, expiry, and access policy. It is not a generic object serializer.

10. Structured concurrency for capabilities and Promises

The current runtime waits for capability settlements before declaring a worker safe.[12][13] A compiler can prevent most unsafe asynchronous shapes statically.

10.1 The problem of detached work

```
JAVASCRIPT
export async function handler(ctx) {
  ctx.cap.audit.write({ event: "started" }); // Promise ignored
  return complete({ ok: true });
}
```

Should the audit complete? May it reject after the worker is returned? Is it part of the result? The current runtime conservatively tracks pending native capability settlements, but arbitrary Promise chains and library tasks can still obscure intent.

10.2 Invocation task scopes

The language profile can require that every asynchronous operation belongs to an invocation task scope:

```
JAVASCRIPT
export async function handler(ctx) {
  await scope.all([
    ctx.cap.audit.write({ event: "started" }),
    ctx.cap.metrics.increment({ name: "attempt" }),
  ]);
  return complete({ ok: true });
}
```

The compiler enforces one of:

- awaited before return;
- joined by a structured combinator;
- explicitly cancelled;
- declared as a host-owned fire-and-forget effect with independent delivery semantics.

No ordinary Promise may outlive the invocation region.

10.3 Deterministic concurrency

`Promise.race` and completion-order-sensitive logic are nondeterministic when capability timing varies. A durable or replayable world must either reject such use, record the winning order as a journal event, or require a deterministic selection rule independent of timing. Durable execution systems solve this by journaling completion order and replaying it.[19]

10.4 Cancellation semantics

Cancellation should be an explicit effect with structured propagation. A timeout cancels child capability calls, closes invocation regions, and marks the worker unsafe unless the VM and host prove quiescence. Compiler-generated task trees make this behavior observable and testable.

11. Secret, evidence, and provenance analysis

Tiny-IDP uses opaque secret handles and native evidence projections. Compiler control can turn that runtime mechanism into a static information-flow discipline.

11.1 Security labels

Values can carry labels such as:

```
TEXT
public
pii.email
sensitive.profile
secret.password
secret.otp
proof.email_verified
proof.webauthn
continuation.safe
log.redacted
```

Labels flow through assignments, object construction, function calls, and returns. Dialect operations declare label transforms. For example:

```
TEXT
normalizeEmail : pii.email -> pii.email
verifyPassword : secret.password x credential.ref -> proof.password_valid
redactEmail    : pii.email -> log.redacted
```

11.2 Enforced sinks

The compiler checks sensitive sinks:

- durable carry accepts only continuation-safe values;
- logs accept public or redacted values;
- presentation redisplay accepts fields whose UI descriptor permits redisplay;
- capability inputs accept only their declared labels;
- commit effects accept proof types, not user-manufactured booleans;
- output schemas reject secret fields.

11.3 Nominal proof types

A JavaScript object `{emailVerified: true}` must not be equivalent to native email evidence. The current object-identity branding pattern already creates unforgeable VM-local handles.[20] A compiler can represent these as nominal resource types:

```
TEXT
EmailProof<challenge_id, address>
PasswordSecret<invocation_region>
InvitationRef<provider_generation>
```

Only native operations can construct them. Their type parameters bind them to the relevant challenge, address, invocation, or provider generation.

11.4 Limits of taint analysis

Information-flow analysis is conservative and incomplete for full JavaScript, especially with reflection, dynamic property access, proxies, exceptions, and implicit flows. The assurance profile should restrict those features at sensitive boundaries. Passing the checker is evidence of compliance with its model, not a claim of side-channel resistance or complete noninterference.

12. Compile effects instead of interpreting arbitrary result objects

Tiny-IDP already treats commit results as inert effect plans executed by Go.[21] A semantic compiler can make effect construction first-class.

12.1 Typed effect operations

Instead of an arbitrary JSON payload:

```
JAVASCRIPT
return { kind: "commit", effects: [{ kind: "create_identity", payload: ... }] };
```

source uses constructors whose signatures are defined by the Tiny-IDP dialect:

JAVASCRIPT

```
return effects.commit([
  identity.create({ subject, email }),
  credential.bindPassword({ identity, password }),
  invitation.consume(invite),
  session.establish({ identity }),
]);
```

The compiler emits a typed effect AST. Unknown keys, illegal resources, or unsupported ordering are rejected before runtime.

12.2 Effect-sequence verification

A dialect verifier can encode rules such as:

- an invitation must be validated before it is consumed;
- a credential binding must target the identity created or loaded in the same commit context;
- a session can be established only after a successful identity effect;
- one-time evidence cannot be consumed twice;
- authorization-code issuance cannot occur in a signup plugin;
- compensatable and non-compensatable effects cannot be mixed without an explicit transaction boundary.

The native committer still revalidates the plan and applies it atomically. Compiler verification reduces the dynamic error surface; it does not replace transaction-time checks.

12.3 Algebraic effect handlers as project composition

At the MIR level, `identity.create` is an abstract effect. Different worlds can install different handlers:

- production handler: SQLite or remote identity service;
- test handler: deterministic in-memory model;
- simulation handler: records an effect trace without applying it;
- migration handler: translates an older effect version;
- model-checking handler: explores success, denial, conflict, and infrastructure failure.

This is a substantial software-design improvement. Application logic becomes independent of the concrete storage and protocol adapters while authority remains explicit.

13. Extract state machines and properties automatically

Once durable suspension points, effects, and outcomes are explicit in MIR, the compiler can produce a finite control graph.

13.1 Static graph

Nodes represent handler blocks and host transitions. Edges carry outcome, schema, evidence, and effect labels.

Analyses can check:

- reachability and dead handlers;
- terminal-state coverage;
- cycles without progress or expiry;
- paths that present a form without CSRF-capable native projection;
- paths that commit without required proof evidence;
- replay-sensitive effects after non-idempotent capability calls;
- continuation schemas that grow across loops;
- unavailable generation migrations.

13.2 Abstract interpretation of opaque compute

Ordinary arithmetic and string manipulation need not be exhaustively modeled. The compiler can summarize a function by its possible outcomes, effects, and predicates. More precise domains can be added for enums, booleans, finite strings, nullability, and schema refinements.

13.3 Property-based test generation

Closed schemas and operation definitions can generate:

- valid and invalid inputs;
- boundary-length strings and byte arrays;
- every enum and outcome case;
- capability failures, timeouts, and malformed results;
- replay and revision conflicts;
- crash injection before and after every effect;
- continuation reload under compatible and incompatible generations.

This resembles IRDL's observation that declarative verifier definitions can generate valid fuzz inputs and can be lowered to analysis systems such as SMT.[6]

Part IV - What direct VM access would enable [Contents](#)

14. Deterministic instruction fuel

Tiny-IDP currently bounds an invocation with a wall-clock timeout and discards a worker after an uncertain interruption.[12] This is correct fail-stop engineering, but wall time is a noisy resource metric. It varies with machine load, garbage collection, capability latency, and host scheduling.

A VM fork could charge deterministic fuel for executed operations:

```
TEXT
fuel -= cost(opcode, operands, allocation_size)
if fuel < 0:
    raise ResourceExhausted
```

14.1 What fuel improves

- reproducible test behavior;
- tenant-independent CPU budgets;
- precise attribution of expensive handlers;
- earlier rejection of loops even on fast machines;
- model-based cost estimation;
- safer fuzzing and symbolic exploration;
- an activation certificate stating a configured execution bound.

Wasmtime exposes a comparable distinction between fuel and epoch interruption: fuel provides deterministic instrumentation, while epoch interruption is a lower-overhead asynchronous deadline mechanism.[22] A Goja fork could use both: fuel for policy and testing, context interruption as the outer emergency brake.

14.2 Cost model design

Charging one unit per bytecode instruction is simple but incomplete. Property lookup, regular expressions, string concatenation, sorting, JSON conversion, and BigInt operations can have input-dependent cost. A useful model needs:

- a base cost per instruction;
- size-sensitive costs for strings, arrays, maps, regex input, and JSON;
- allocation cost;
- host-call cost charged separately by capability policy;
- periodic deadline checks independent of fuel.

The model should be deterministic and conservative, not a promise of exact CPU cycles.

14.3 Static and dynamic bounds

A restricted profile can statically prove bounds for loops over bounded schema collections. Full JavaScript cannot generally provide a useful static termination bound. The design should combine:

- admission-time rejection of obviously unbounded constructs in high-assurance handlers;
- compiler estimates for bounded loops;
- runtime fuel for dynamic paths;
- wall-clock interruption for engine and host anomalies.

This resembles the eBPF model: programs are admitted only after verification under finite complexity limits, while the runtime remains simple.[23]

15. Allocation and retained-heap accounting

A timeout does not prevent a script from allocating a large object graph quickly. A custom VM can account for:

- object, array, string, symbol, and buffer allocations;
- backing-store growth;
- compiled-code and source-map memory;
- Promise jobs and reaction records;

- retained heap after invocation;
- host wrappers and native handles.

QuickJS exposes explicit runtime memory limits, custom allocation hooks, maximum stack size, and an interrupt handler.[24] These are useful reference capabilities even though its C implementation and bytecode format are not directly transferable to Goja.

15.1 Region accounting

The most useful model is not one global byte counter but regions:

```
TEXT
runtime region    - shared frozen intrinsics and module code
module region    - top-level immutable plugin data
invocation region - ordinary transient JS allocations
secret region    - opaque native buffers and handles
continuation region- serialized, schema-checked public carry
```

At invocation completion, the runtime can require the invocation region to become unreachable except for explicitly permitted cache objects. If not, the worker is discarded or the retained bytes are charged to a module-state quota.

15.2 Why Go heap accounting is difficult

Goja objects live on the Go heap, where exact object size and liveness are controlled by Go's garbage collector. Instrumentation can count logical allocation sizes but cannot cheaply know exact retained physical memory at every point. A practical implementation should document that quotas are semantic approximations, use periodic heap sampling for operations, and retain process-level memory limits as an outer boundary.

16. Job-queue control and deterministic Promise semantics

ECMAScript defines Jobs and leaves scheduling to the host, subject to ordering and run-to-completion constraints.[3] Goja deliberately expects the embedding application to own the event loop.[9] Tiny-IDP already routes native capability settlement through the runtime owner.

Direct VM access could expose a first-class job queue with:

- job IDs and source locations;
- parent task and invocation region;
- deterministic sequence numbers;
- pending-job count and quiescence proof;
- rejection tracking as an activation/runtime error;
- cancellation propagation;
- a policy hook before enqueue and before execution;
- journal integration for durable concurrency.

16.1 Quiescence as a worker-safety theorem

A worker should be reusable only if:

```
TEXT
no active invocation capability
and no pending invocation job
and no unresolved invocation Promise
and no live invocation-scoped host resource
and interrupt flag is clear
and result passed its contract
```

The current implementation approximates this with capability settlement tracking and fail-stop disposal.[12][13] A VM-level queue can make the proof complete for JavaScript-created microtasks as well.

16.2 Replayable scheduling

For deterministic tests, the host can execute jobs in a controlled order. For durable workflows, any result whose meaning depends on external completion order must be journaled. A trace could record:

TEXT

```

job 17 enqueued by cap.community.lookup
job 18 enqueued by timer.deadline
job 18 selected first
job 17 cancelled

```

Replay consumes the same selection record rather than racing live operations.

17. Internal provenance and unforgeable value classes

Tiny-IDP currently brands blank Goja objects by pointer identity in host maps.[20] That pattern is strong but localized. VM access could add an internal, non-enumerable provenance slot to values or objects.

Possible tags include:

TEXT

```

Origin.NativeEvidence(type, id, generation)
Origin.Secret(region, kind)
Origin.Capability(invocation, id, version)
Origin.SchemaValidated(schema, version)
Origin.DurableReference(kind, id)

```

JavaScript reflection cannot observe, copy, or synthesize these tags. VM operations define how provenance propagates or is erased.

17.1 Benefits

- native proof values remain nominal even through wrapper objects;
- the encoder can reject any value transitively containing a secret tag;
- logs can automatically redact tagged values;
- capability functions can verify invocation epoch without a Go-side closure map lookup;
- cross-realm and cross-worker transfers can be rejected precisely;
- traces can attribute data origins without exposing payloads.

17.2 Risks

A provenance system inserted into a dynamic object model can become complex and expensive. Tags do not automatically follow all implicit information flows. The implementation must avoid suggesting stronger confidentiality guarantees than it proves. Start with nominal resource classes and sink checks, not universal taint propagation.

18. Semantic tracing, debugging, and explainability

Interpreter access makes traces precise and cheap enough to become part of the platform contract.

A semantic trace event can contain:

TEXT

```

artifact, world, handler, source span, IR operation,
capability/effect ID, outcome kind, fuel delta,
allocation delta, task ID, continuation revision,
redacted value shape, diagnostic correlation ID

```

18.1 Trace levels

- **audit trace:** capability calls, native evidence use, presentations, challenges, effects, outcomes, generation and revision changes;
- **debug trace:** control-flow blocks, selected branches, Promise jobs, catches, and source spans;
- **profile trace:** instruction and allocation counters;
- **verification trace:** abstract-state transitions and assertion checks.

Payload values should be absent by default. Schema and provenance metadata determines whether a value may be sampled, hashed, or redacted.

18.2 Deterministic replay debugger

Given a signed artifact, input, fake capability transcript, virtual clock, random seed, and scheduling journal, a developer can replay an invocation step by step. This is substantially more useful than a raw Goja stack trace because it shows domain operations and effect boundaries.

18.3 Source mapping through transformations

Every MIR operation must preserve a source span and transformation provenance. Diagnostics should be able to explain:

```
TEXT
source signup.js:31:16
  lowered by durable-await@1.2
to handler community.signup.after_form@2 block 4
which emitted continuation schema sha256:...
```

Without this, compiler-generated handlers become operationally opaque.

19. Runtime snapshots and reset: useful, but not durable control flow

A custom VM might support a frozen activation snapshot: initialize intrinsics and module definitions once, then clone or restore the runtime for each worker. This could reduce startup cost and eliminate state leakage.

19.1 Safe snapshot candidates

- parsed or compiled program;
- immutable strings and constants;
- frozen intrinsic graph;
- frozen module exports;
- callback registry identity metadata.

19.2 Unsafe snapshot candidates

- native file/network/store handles;
- event-loop state;
- pending Promises;
- invocation capabilities;
- secret buffers;
- goroutine synchronization objects;
- pointers into external Go objects;
- active stacks or exception frames.

19.3 Reset versus clone

A practical initial feature is **hard reset**: discard the runtime and recreate it from a compiled artifact. Tiny-IDP already does this after unsafe invocation.[25] A later snapshot implementation must prove that cloned values do not retain native pointers or hidden mutable state.

20. Why VM-frame persistence is the wrong continuation model

Goja internally has suspension and resumption machinery. The pinned `vm.go` shows an execution context containing a program pointer, lexical stash, private environment, result, program counter, stack base, arguments, and related stacks. Suspension copies stack values and portions of try, iterator, and reference stacks; resumption restores them into the VM.[26]

This is appropriate for an in-memory generator or async function. It is not a durable serialization format.

20.1 Persistence hazards

1. **Engine coupling.** A field layout or opcode change invalidates all records.
2. **Heap closure.** Stack values point into a potentially large cyclic object graph.
3. **Native pointers.** Host objects and functions may reference Go memory, stores, requests, or secrets.

4. **Garbage-collector invariants.** Restoring arbitrary graphs must cooperate with Go and Goja object lifecycles.
5. **Security review.** A serialized heap is difficult to schema-check, redact, bind, or migrate.
6. **Replay ambiguity.** Pending jobs and external calls may have partially completed.
7. **Hot reload.** Program counters have meaning only for the exact compiled instruction stream.
8. **Cryptographic binding.** Authenticating a large opaque VM image does not make its internal authority safe.
9. **Operational size.** A small form wait could persist megabytes of irrelevant objects.
10. **Inspection and repair.** Operators cannot meaningfully inspect or patch the control state.

20.2 Correct use of VM suspension knowledge

Interpreter access is still valuable. It reveals the language constructs and data that must be represented by the compiler transform. It can help differential-test the transformed state machine against ordinary in-memory async execution. It should not become the persisted representation.

Part V - A project-independent semantic intermediate representation Contents

21. Why an MIR is the strategic center

A source transform that emits more JavaScript can deliver durable `await` quickly, but it still leaves long-lived semantics encoded in generated source and host conventions. A stable MIR provides a better boundary.

The MIR should be:

- deterministic and canonically serializable;
- independent of Goja value and bytecode formats;
- small enough to specify formally;
- expressive enough for control flow, bounded computation, capabilities, effects, outcomes, and suspension;
- extensible through versioned dialects without weakening core invariants;
- source-mapped back to JavaScript;
- executable by a deliberately simple reference interpreter;
- lowerable to optimized backends.

CEL offers a useful control-plane/data-plane precedent: parse and type-check ahead of time, serialize a canonical checked AST, then evaluate it against an explicit environment.[8] Malleable JS needs statements, effects, and continuations beyond CEL's expression model, but the artifact discipline is similar.

22. Core MIR concepts

A minimal core could contain the following.

22.1 Values and types

```
TEXT
Bool, I64, U64, String<N>, Bytes<N>
Option<T>, Result<T,E>
List<T,N>, Record<ID>, Variant<ID>
Resource<Dialect, Kind, Region, Parameters...>
Function<Input, Effects, Output>
```

The absence of an unrestricted `Any` type is important. Dynamic JSON may exist only as a schema-bound `Json<SchemaID>` value with closed access operations.

22.2 Regions and lifetimes

```
TEXT
static          - artifact constants and frozen module definitions
invocation      - values valid only during one request
secret          - opaque native secret resources
transaction     - resources valid during one native commit
continuation    - serializable values/references allowed in durable carry
```

A resource type names its region. The verifier rejects a value escaping to a longer-lived region.

22.3 Control flow

The core uses explicit blocks and terminators:

```
TEXT
branch condition then blockA else blockB
return value
throw error
suspend operation resume block carry values
switch variant cases
```

No implicit browser suspension exists after lowering.

22.4 Effects

Effects are named dialect operations with typed operands and results:

TEXT

```
%member = cap.call @community.lookup(%email)
%page    = ui.present @signup.form(%model)
suspend %page resume ^after_submit carry(%member_ref)
commit [identity.create(...), invitation.consume(...)]
```

Each operation declares interfaces such as:

TEXT

```
Pure
MaySuspend
CapabilityCall
CommitEffect
ProducesEvidence
ConsumesResource
DeterministicGivenJournal
RedactionAware
```

This follows MLIR's insight that generic analyses should query operation interfaces rather than hard-code every dialect operation.[27]

22.5 Outcomes

Closed outcome variants remain part of the core because they define the host boundary:

TEXT

```
Continue<Handler, Input>
Present<Page, Continuation>
Challenge<Request, Continuation>
Commit<EffectSequence>
Complete<Value>
Deny<Code>
Skip<Code>
Error<Diagnostic>
```

23. An illustrative MIR

The earlier signup example could lower to:

TEXT

```
module @community.signup version 4
world @tinyidp.policy version 2

func @signup.entry(%input: !schema<SignupStart>)
  effects [cap.invites.inspect, ui.present]
  outcomes [present, deny, error]
{
^entry:
  %code = schema.get %input["inviteCode"]
  %invite = cap.call @invites.inspect(%code)
  %ok = variant.is %invite, #valid
  branch %ok ^show_form ^deny_invalid

^show_form:
  %email = variant.get %invite, #valid.email
  %ref = variant.get %invite, #valid.ref
  %page = ui.page @signup.form { email = %email }
  suspend present %page
  resume @signup.after_form
  carry !schema<AfterFormCarry> { inviteRef = %ref }

^deny_invalid:
  return deny #invalid_invitation
}

func @signup.after_form(
  %event: !schema<SignupFormPost>,
  %carry: !schema<AfterFormCarry>
) effects [challenge.email]
  outcomes [challenge, deny, error]
{
  %email = schema.get %event["email"]
  %challenge = challenge.start @email_code(%email)
  suspend challenge %challenge
}
```

```

resume @signup.after_email
carry !schema<AfterEmailCarry> { email = %email,
                                inviteRef = %carry.inviteRef }
}

```

This representation can be validated without executing JavaScript. It also gives the production interpreter a narrow set of terminators and effect operations.

24. Reference interpreter and optimized backends

24.1 Reference interpreter

The reference interpreter should be intentionally simple, deterministic, and unsuitable for high-throughput production if simplicity aids review. It executes MIR against an abstract host interface and produces an effect trace.

Its purposes are:

- executable semantics;
- differential oracle for optimized backends;
- model-checking integration;
- deterministic tests;
- formalization alignment;
- artifact inspection and debugging.

24.2 Goja backend

Pure-compute regions can continue to execute in Goja. The compiler can either:

- emit JavaScript functions for compute blocks and call them from the MIR interpreter;
- preserve original callback bodies but surround effect/suspension boundaries with generated code;
- lower the entire accepted subset to Goja AST/program objects.

The backend must demonstrate trace equivalence with the MIR reference interpreter for accepted programs, modulo permitted implementation details.

24.3 Native Go backend

For hot paths or especially critical handlers, MIR can compile to generated Go. This does not change source-language semantics and allows comparison among three executions: reference MIR, Goja, and generated Go.

24.4 Analysis backends

The same artifact can feed:

- graph visualizers;
- reachability and liveness analyses;
- SMT encodings for bounded properties;
- schema and API documentation;
- test-case generators;
- effect and capability inventories;
- upgrade compatibility checkers.

25. Proof-carrying activation artifacts

A compiled package should include an activation certificate:

```

TEXT
Certificate {
  source_hash
  frontend_name_and_version
  language_profile
  world_id_and_version
  dialect_ids_and_versions
  pass_pipeline_hash
  canonical_mir_hash
  callback_and_handler_ids
  inferred_capabilities
}

```

```

inferred_effects
continuation_schema_hashes
resource_policy
verifier_results
test_and_differential_results
signature
}

```

The production runtime does not trust prose claims. It verifies hashes, signatures, world compatibility, dialect availability, and certificate predicates before loading code.

25.1 Certificates are not proofs by themselves

A certificate is useful only if its checker is small and trustworthy. Some fields are attestations that a tool ran; others can contain machine-checkable witnesses. The long-term goal is to minimize the trusted checker and make critical claims independently reproducible.

25.2 Activation identity

Tiny-IDP currently fingerprints source, canonical program, callback registry, and schemas. MIR adds semantic identities:

```

TEXT
source identity      - exact author text
frontend identity    - parser/profile/transform versions
semantic identity    - canonical MIR
world identity       - allowed imports and policy
continuation identity- handler and carry schema versions
backend identity     - Goja or native code generation version

```

A continuation should bind primarily to semantic and continuation identity, while source/backend identities remain available for audit and compatibility policy.

26. Formal verification strategy

26.1 What to formalize first

A practical formal core should cover:

- value and resource types;
- region/lifetime rules;
- block and terminator semantics;
- effect traces;
- suspension and resumption;
- closed outcomes;
- verifier judgments;
- world import satisfaction;
- continuation carry serialization.

26.2 Candidate theorems

1. **Region safety:** a well-verified program cannot place invocation or secret resources in continuation carry or ordinary output.
2. **Capability confinement:** every emitted capability effect belongs to the function's inferred effect row and the selected world's imports.
3. **Outcome closure:** execution of a well-verified function terminates at a declared core outcome or resource error, not an untyped host return.
4. **Continuation type preservation:** resuming a valid continuation under the same semantic generation supplies values matching the destination block types.
5. **Effect-plan well-formedness:** a committed plan satisfies dialect sequencing constraints before reaching the native transaction handler.
6. **Lowering trace preservation:** for the accepted source subset, compiler lowering preserves observable outcomes and abstract effect traces.

26.3 Production correspondence

Cedar's methodology is instructive: formally prove a validator property in Lean, then differentially test the production Rust validator against the formal implementation.[7] Malleable JS can similarly compare:

- formal MIR semantics;
- executable reference interpreter;
- Goja backend;
- generated Go backend;
- native transition interpreter.

26.4 Why not verify modern JavaScript first

JSCert and KJS show that rigorous JavaScript semantics are possible.[10][11] They target older ECMAScript cores and required substantial research effort. Modern ECMAScript continues to evolve, and a complete engine must track thousands of detailed semantic interactions and the official Test262 suite.[3][4] Verification effort is better concentrated on the small accepted profile, its lowering, and the identity-specific effect system.

Part VI - Making the compiler malleable across projects Contents

27. Dialects, worlds, passes, and backends

The extensibility model should separate four concepts that are often collapsed into a plugin API.

27.1 Dialect

A dialect defines domain semantics:

```
TEXT
Dialect {
  package ID and semantic version
  value and resource types
  operations and terminators
  effect atoms
  operation interfaces and traits
  local verification rules
  canonical serialization
  source-language bindings
  lowering and rewrite rules
  documentation metadata
  test and fuzz generators
  compatibility and migration declarations
}
```

Examples:

```
TEXT
core.control
core.schema
core.resource
tinyidp.workflow
tinyidp.presentation
tinyidp.challenge
tinyidp.commit
approval.human
agent.tools
migration.database
notification.delivery
```

MLIR demonstrates why this division scales: dialects add operations and types; declarative definitions reduce boilerplate; interfaces let generic analyses operate without knowing every concrete operation; extensible dialects can even be registered at runtime.[6][27]

27.2 World

A world is the external contract for one class of plugin. It selects exact dialect versions, imported capabilities, exported entry points, resource limits, language profile, and trust policy.

The WebAssembly Component Model's WIT worlds are a strong analogy. A world declares what a component provides and what it requires; if a secret-store interface is not imported, the component cannot access it through that boundary.[28] Malleable JS worlds should provide the same immediately reviewable authority inventory.

Example:

```
TEXT
world tinyidp.signup@4 {
  profile assurance-js@1.3

  use core.control@1
  use core.schema@2
  use tinyidp.workflow@3
  use tinyidp.presentation@2
  use tinyidp.challenge@2
  use tinyidp.commit@4

  import cap.invitation.inspect@1
  import cap.community.lookup@2

  export workflow signup

  limits {
```

```

    fuel = 200000
    allocation_bytes = 1048576
    capability_calls = 8
    continuation_bytes = 8192
    durable_suspensions_per_path = 6
  }
}

```

27.3 Pass

A pass is a deterministic transformation or analysis over declared input and output dialects:

```

TEXT
parse-js
resolve-modules
check-assurance-profile
infer-types
infer-effects
lower-durable-await
infer-continuation-carry
lower-js-control-to-core
verify-regions
canonicalize-effects
extract-state-graph
emit-goja
emit-certificate

```

Every pass has a version, configuration hash, declared preserved properties, and diagnostics. The pipeline hash becomes part of the artifact certificate.

27.4 Backend

A backend consumes verified MIR:

- Goja execution;
- native reference interpreter;
- generated Go;
- WebAssembly component;
- graph/documentation generator;
- symbolic or bounded model checker;
- deterministic simulator.

A project can therefore reuse the source language and compiler infrastructure without sharing its runtime deployment choice.

28. Declarative dialect definitions

Most project extensions should be data, not arbitrary compiler code. A declarative operation definition might look like:

```

YAML
package: tinyidp.challenge
version: 2.1.0

operations:
- name: email-code
  kind: durable-effect
  inputs:
    address: pii.email
    template: enum[email-verification, password-reset]
  result: resource<email-proof, continuation>
  effects:
    - challenge.email.send
    - workflow.suspend
  traits:
    - produces-evidence
    - redaction-aware
    - replay-journaled
  verify:
    - "template == password-reset implies world.has(reset-flow)"
    - "address.label includes pii.email"
  lowering:
    tinyidp-native: challenge.email.v2

```

From this single definition, tools can generate:

- JavaScript/TypeScript bindings;
- MIR parser/printer support;
- runtime dispatch stubs;
- validation and diagnostic scaffolding;
- API documentation;
- fake capability implementations;
- property-based generators;
- redaction rules;
- effect inventories;
- LSP completion metadata.

IRDL's goals of portability, introspection, runtime declaration, reliability, fuzzer generation, and SMT lowering are directly relevant.[6]

28.1 Declarative rules have limits

Complex verification will sometimes require native code or a theorem-prover model. The extension system should make that escalation explicit rather than embedding arbitrary callbacks in every operation definition.

29. Extension trust tiers

Malleability expands the trusted computing base unless extensions are classified.

Tier 0 - Data-only catalogs

Schemas, enums, documentation, operation signatures, and compatibility metadata. These are parsed by the core and cannot execute compiler code.

Tier 1 - Declarative verifier rules

A small total expression language evaluates constraints over operation attributes and types. CEL-like restrictions are appropriate: mutation-free, terminating, typed, and canonically serialized.[8]

Tier 2 - Verified or audited compiler modules

Native passes implement transformations that cannot be expressed declaratively. They must declare dependencies and preserved invariants, pass differential tests, and be included in the signed compiler distribution.

Tier 3 - Experimental VM extensions

New opcodes, host-object classes, compiler intrinsics, or scheduler behavior. These are the highest-risk extensions and should never be loadable from an ordinary application plugin.

29.1 Control-plane compilation

Production services should not dynamically load arbitrary compiler plugins. Compilation occurs in a controlled build or activation service. The result is a signed, self-contained semantic artifact. Runtime nodes need only the small artifact checker, selected dialect runtime handlers, and backend.

29.2 Governance

Each dialect package needs:

- owner and security contact;
- semantic versioning policy;
- compatibility test suite;
- deprecation schedule;
- stable operation and type IDs;
- migration policy for persistent state;
- threat model and authority inventory;
- supported compiler and world versions.

Without governance, extensible compiler infrastructure becomes a collection of undocumented special cases.

30. Tiny-IDP as a dialect and world pack

The present Tiny-IDP packages map cleanly onto dialect responsibilities.

Existing responsibility	Proposed dialect/platform role
idpprogram.Schema	core.schema types and closed-record verifier
LambdaSpec	generated function type/effect/resource contract
closed Outcome family	core host-boundary variants and terminators
CapabilityRequirement	world imports and capability effect atoms
presentation field/action brands	nominal UI descriptor resources
secret handles	invocation-region secret resources
challenge requests/evidence	challenge operations and native proof resources
commit EffectPlan	typed Tiny-IDP commit operations
workflow continuation	core suspension record specialized by Tiny-IDP bindings
verification plans	declarative scenario/test dialect
provider adapters	world-specific native effect handlers

The identity-specific invariants remain in Go and in Tiny-IDP dialect verifiers. The generic compiler knows only resources, effects, regions, suspension, schemas, worlds, and operation interfaces.

31. Other project families

The same substrate could support projects with similar requirements without importing identity-specific concepts.

31.1 Human approval workflows

A deployment or financial-approval world could define:

```
TEXT
request approval
present review UI
wait for signed decision
execute bounded operation
record immutable audit event
```

Durable `await` and continuation carry analysis apply directly. Approval evidence becomes a native nominal resource.

31.2 Agent tool orchestration

An AI-agent world can expose a finite tool catalog, explicit information labels, approval gates, cost budgets, and durable task waits. The compiler can reject dynamic tool names, detached calls, unbounded parallel fan-out, or secret-to-model flows. The same effect trace supports simulation and human review.

31.3 Database migration orchestration

A migration world can model schema versions, locks, reversible and irreversible steps, checks, and compensation. A compiler can ensure that irreversible effects do not occur before required validation and that resumable checkpoints contain only stable references.

31.4 Notification and delivery policy

A notification world can select templates, audiences, channels, throttles, and fallback behavior. Plugins never receive raw network clients; they emit typed delivery effects. Test backends render the delivery plan without sending messages.

31.5 Build and configuration systems

A deterministic world can resemble Starlark: no ambient time/network/filesystem, frozen module outputs, finite loops, and structured data as the result.[15] JavaScript syntax may improve familiarity while the selected profile removes application-language hazards.

31.6 Edge authorization and request policy

For simple predicates, a world may lower to CEL or Cedar instead of Goja. Malleable JS does not need to be the evaluator for every subproblem. It can use the same world/dialect/certificate infrastructure to embed a smaller purpose-built language where appropriate.

32. Multi-frontend architecture

Once MIR is stable, JavaScript need not be the only source language.

Possible front ends include:

- a TypeScript syntactic subset with erased or checked annotations;
- a declarative YAML/JSON builder for simple programs;
- CEL expressions embedded in branches;
- generated programs from a visual workflow editor;
- Go or Rust macros for native teams;
- imported WebAssembly components whose WIT world maps to a Malleable JS world.

This does not imply semantic fragmentation. Every front end must lower to the same MIR and satisfy the same verifier. JavaScript remains the primary ergonomic language, not the durable semantic format.

Part VII - The plugin-writer experience [Contents](#)

33. From manual contracts to inferred contracts

A current-style handler declaration may repeat identity, schemas, capabilities, outcomes, effects, and budgets around a callback. A compiler-oriented API can move those facts to module and world declarations.

33.1 Proposed source

```
JAVASCRIPT
import {
  workflow,
  durable,
  effects,
  schema,
} from "world:tinyidp.signup@4";

const SignupForm = schema.form({
  email: schema.email({ redisplay: true }),
  displayName: schema.string({ max: 120, redisplay: true }),
  password: schema.password(),
});

export default workflow("signup", async ctx => {
  const invitation = await ctx.invitation.inspect(ctx.input.code);

  if (invitation.kind !== "valid") {
    return effects.deny("invalid_invitation");
  }

  const form = await durable.form(SignupForm, {
    email: invitation.email,
  });

  const proof = await durable.emailCode(form.email);

  return effects.commit([
    ctx.identity.create({
      email: form.email,
      displayName: form.displayName,
      verifiedBy: proof,
    }),
    ctx.credential.bindPassword(form.password),
    ctx.invitation.consume(invitation.ref),
    ctx.session.establish(),
  ]);
});
```

33.2 Generated contract

```
TEXT
workflow signup@4
handlers:
  signup.entry
  signup.after_form
  signup.after_email

imports:
  invitation.inspect@1

outcomes:
  deny, present, challenge, commit, error

effects:
  identity.create@2
  credential.bind-password@2
  invitation.consume@1
  session.establish@1

continuations:
  after_form carry SignupAfterFormCarry@sha256:...
  after_email carry SignupAfterEmailCarry@sha256:...

resources:
  secret.password scoped to after_form invocation
  proof.email scoped to challenge and commit
```

The author can inspect and approve this manifest during review.

34. Diagnostics as the primary usability feature

Compiler sophistication is useful only if errors explain domain meaning.

34.1 Capability error

TEXT

MJS-CAP-0042 Undeclared capability

``community.lookup`` is reachable from ``signup.after_form``, but the selected world does not import it.

Add the import to the world, remove the call, or route the lookup through an approved higher-level operation.

34.2 Secret escape

TEXT

MJS-REG-0117 Secret crosses durable boundary

``form.password`` has type `Secret<Password, invocation#2>` and is used after ``await durable.emailCode(...)``. The email-code wait resumes in a new invocation, where this secret no longer exists.

Move password verification/binding before the durable wait, or request the password again in a later native presentation.

34.3 Nondeterminism

TEXT

MJS-DET-0203 Unjournalled time affects durable control flow

The result of ``Date.now()`` selects a branch that can reach a durable suspension. Use ``ctx.clock.now()`` so the value is supplied and journaled by the host.

34.4 Detached asynchronous task

TEXT

MJS-TASK-0308 Promise escapes invocation task scope

The Promise returned by ``ctx.audit.write(...)`` is neither awaited, joined, cancelled, nor declared as a host-owned delivery effect.

34.5 Upgrade compatibility

TEXT

MJS-MIG-0401 Incompatible continuation schema change

Handler ``signup.after_email@2`` removed field ``inviteRef`` and changed ``email`` from `EmailAddress@1` to `VerifiedEmail@2`. There are 184 active continuation records under the prior schema.

35. IDE and review tooling

A language server can expose domain-specific information derived from MIR.

35.1 Editor features

- completion only for capabilities imported by the selected world;
- hover display of effects, labels, regions, and versioned native operation IDs;
- go-to-definition from generated handlers to source `await` sites;
- inline lens showing inferred outcomes and continuation carry;
- state-machine visualization;
- commit-effect sequence explanation;
- warning when a change alters semantic or continuation identity;
- quick fixes for joining Promises or converting values to durable references;

- generated schema examples and form previews.

35.2 Review views

Code review should include semantic diffs:

```
TEXT
+ imports cap.community.lookup@2
+ durable state signup.after_recovery
~ continuation carry after_form: 72 bytes -> 416 bytes
+ effect credential.bind-recovery@1
- denial outcome member_exists
~ maximum fuel estimate 42k -> 91k
```

A small source diff can have a large authority or persistence impact. Semantic diffs make that impact visible.

36. Testing and simulation

36.1 Virtual host

Tests instantiate a world with deterministic handlers:

```
JAVASCRIPT
const result = await test.run(signup, {
  input: { code: "INVITE-1" },
  capabilities: {
    "invitation.inspect": fake.return({
      kind: "valid",
      email: "a@example.test",
      ref: fake.resource("invite-1"),
    }),
  },
  durable: [
    form.submit({ email: "a@example.test", displayName: "A", password: secret("x") }),
    challenge.succeed("email-code"),
  ],
});

expect(result.effects).toEqual([
  "identity.create",
  "credential.bind-password",
  "invitation.consume",
  "session.establish",
]);
```

36.2 Fault schedules

The test runner can inject failure at every semantic boundary:

```
TEXT
before capability call
after capability completion before Promise settlement
before continuation insert
after continuation insert before response
before challenge delivery
after challenge proof before revision advance
before commit
between each native effect
before transaction commit
after transaction commit before terminal response
```

Expected idempotency, replay, and worker-disposal behavior can be generated from operation traits.

36.3 Model-based coverage

Coverage should include:

- source branches;
- MIR blocks and outcomes;
- capability/effect variants;
- continuation states;
- verifier rule cases;
- failure and retry boundaries;
- generation migration paths.

This is more meaningful than line coverage alone.

37. Packaging, linking, and reusable libraries

A package declares:

```
TEXT
package identity.example/recovery@2.3.1
requires world-interface tinyidp/recovery@2
exports workflow password-recovery
imports library identity.example/common-validation@1
```

Libraries publish MIR summaries and source maps. The linker resolves world imports, dialect versions, resource types, and effects. A library cannot gain authority merely because its caller has it; capability values must be passed or its effect summary must be satisfied explicitly.

37.1 Attenuated capabilities

A caller may pass a narrowed wrapper:

```
JAVASCRIPT
const lookupByEmail = attenuate(ctx.community.lookup, {
  fields: ["exists"],
  maxCalls: 1,
});

return commonCheck({ lookupByEmail });
```

The compiler models attenuation as a new capability type whose result schema and budget are narrower than the original.

37.2 Dependency review

A lockfile should include semantic hashes and authority summaries, not only source package versions:

```
TEXT
common-validation@1.4.2
mir sha256:...
effects: [cap.community.lookup]
resources: none
durable suspension: none
profile: assurance-js@1.3
```

38. Semantic hot reload and migration

The current Tiny-IDP design correctly pins continuations to generation identity.[14] A compiler platform can add a migration language.

38.1 Compatibility categories

- **backend-only change:** same MIR and schemas; safe to replace workers;
- **source-only equivalent change:** different source, same canonical MIR;
- **compatible semantic change:** new branches/effects that do not affect existing continuation handlers;
- **continuation migration:** old carry schema transformed to new schema under an explicit total function;
- **breaking change:** old generation must remain resident or active continuations must be revoked.

38.2 Migration functions

Migration code should be pure, terminating, and operate only on public carry and native references:

```
JAVASCRIPT
migrate("signup.after_email", from(2), to(3), old => ({
  email: normalizeEmail(old.email),
  inviteRef: old.inviteRef,
  locale: "en-US",
}));
```

The compiler verifies totality over the old schema where feasible, forbids capabilities and secrets, and emits a migration artifact separately from request handlers.

Part VIII - Evaluation, risks, and roadmap Contents

39. Option matrix

Criterion	Public Goja	AST/front end	Goja fork	Own MIR/interpreter	Full JS engine
Initial engineering cost	Low	Medium	Medium-high	High	Extreme
Ongoing upstream burden	Low	Medium	High	Medium-high	Extreme
Static capability/effect inference	Limited	Strong	Strong	Strongest	Strong
Durable await lowering	Source wrapper only	Strong	Strong	Strongest	Strong
Deterministic instruction fuel	No	Approximate	Strong	Strong	Strong
Allocation accounting	Weak	Weak	Medium/strong	Strong for MIR values	Strong
Formal verification target	Host contract	Profile/passes	VM subset	Small core	Entire language/runtime
JavaScript compatibility	Goja level	Goja level/profile subset	Goja level	Source-profile level	Must be implemented
Multi-project extensibility	Host libraries	Good	Good	Excellent	Potentially excellent
Risk of durable-engine coupling	Low	Low	Medium if misused	Low	High if bytecode/frames persisted
Recommended role	Baseline	Build now	Add selectively	Strategic core	Avoid initially

40. Principal risks and mitigations

40.1 Goja AST instability

Goja's AST package explicitly warns that parser and AST interfaces remain works in progress.[29]

Mitigation: isolate Goja AST access behind one adapter; immediately lower to a stable internal syntax tree; pin the Goja commit; maintain parser conformance fixtures; avoid allowing dialects to depend directly on Goja node types.

40.2 Fork divergence

VM instrumentation can conflict with upstream compiler and runtime changes.

Mitigation: keep patches generic and small; maintain a continuously rebased branch; upstream hooks where possible; run upstream Goja tests and Test262; version the backend independently from semantic artifacts.

40.3 Nonstandard JavaScript syntax

Custom syntax burdens editors, formatters, linters, source maps, and developers.

Mitigation: begin with standard JavaScript plus imported intrinsics, JSDoc annotations, or TypeScript-compatible declarations. Introduce syntax only when it removes substantial accidental complexity, and implement it in the language server and formatter first.

40.4 False confidence from static analysis

Full JavaScript frustrates precise analysis.

Mitigation: make assurance profiles explicit; reject dynamic features at security boundaries; report inferred approximations and unknowns; retain runtime validation; distinguish proved, checked, tested, and assumed properties in certificates.

40.5 Compiler plugins expand the trusted base

An arbitrary extension pass can falsify types or remove checks.

Mitigation: use declarative dialects by default; sign trusted pass distributions; verify MIR again after every untrusted transformation; compile in a controlled service; keep the production certificate checker independent.

40.6 Resource metering mismatch

Fuel and logical allocation may not track real CPU and memory perfectly.

Mitigation: use layered limits: static restrictions, fuel, semantic allocation quotas, wall time, process memory, pool saturation, and OS/container limits.

40.7 In-process hostile code

A language subset and capabilities do not eliminate engine vulnerabilities or side channels.

Mitigation: define the trust model. Operator-authored policy may run in-process. Third-party or adversarial code should run in a separate process, sandbox, or WebAssembly runtime with narrowly typed IPC.

40.8 Over-generalization

A cross-project platform can delay the concrete Tiny-IDP product.

Mitigation: derive the core only from at least two real worlds; keep Tiny-IDP as the reference customer; avoid abstractions without a verifier, lowering, or tooling use case; ship vertical slices.

41. Staged implementation roadmap

Phase 0 - Threat model and semantic contract

Define:

- trusted operator scripts versus third-party plugins;
- which properties are enforced statically, dynamically, or operationally;
- accepted JavaScript profile;
- artifact and continuation compatibility policy;
- compiler/dialect governance;
- out-of-process isolation requirements.

Deliverable: an architecture decision record and conformance matrix.

Phase 1 - Front end above stock Goja

Build:

- pinned Goja parser adapter;
- stable internal syntax tree;
- source locations and diagnostics;
- profile checker;
- capability/effect/outcome inference;
- generated LambdaSpec and semantic manifest;
- frozen-primordial and deterministic-world initialization.

No new syntax and no VM fork.

Phase 2 - Durable await prototype

Implement one durable operation, such as `durable.form`:

- identify suspension points;
- split source into generated handlers;
- compute live carry;
- reject nonserializable and secret captures;
- generate continuation schemas and stable IDs;
- differential-test transformed execution against explicit hand-written handlers.

Then add challenge waits and external callbacks.

Phase 3 - Structured concurrency and deterministic host

Add:

- task-scope analysis;
- detached-Promise rejection;
- virtual clock and random effects;
- journaled race/select primitives;
- semantic traces;
- improved worker quiescence checks using public APIs where possible.

Phase 4 - Introduce MIR and reference interpreter

Create project-independent packages, tentatively:

```
TEXT
mjs/syntax
mjs/ir
mjs/types
mjs/effects
mjs/verify
mjs/dialect
mjs/world
mjs/cert
mjs/interp
mjs/backend/goja
mjs/tools/lsp
```

Lower the existing Tiny-IDP compiler output to MIR without changing production behavior. Run the reference interpreter in tests and shadow mode.

Phase 5 - Dialect SDK

Extract Tiny-IDP operations into declarative dialect packages. Generate bindings, docs, fake hosts, and tests. Implement a second small world, such as human approval or notification policy, to test generality.

Phase 6 - Selective Goja instrumentation

Only after measurement identifies gaps, add:

- instruction fuel;
- logical allocation accounting;
- explicit microtask/job hooks;
- provenance tags for native resources;
- semantic trace probes;
- stronger reset/snapshot support.

Maintain each feature behind a generic upstream-oriented interface.

Phase 7 - Formal assurance

- formalize core MIR semantics and typing;
- prove region safety and capability confinement;
- verify the continuation carry checker;
- prove selected lowering rules;
- differential-test formal/reference/Goja implementations;
- add machine-checkable witnesses to activation certificates.

Phase 8 - Isolation and additional backends

- optional WebAssembly component backend;
- out-of-process plugin runner;
- generated Go backend;
- model checker and symbolic executor;
- multi-language front ends.

Phase 9 - Reconsider a full engine

A complete new JavaScript implementation is justified only if all are true:

- multiple strategic products require the same language platform;
- the accepted compatibility target is explicitly bounded;
- a dedicated runtime/compiler team exists;
- ongoing Test262, performance, security, and tooling maintenance is funded;
- selective Goja modification cannot supply required guarantees;
- owning the engine is itself a product advantage.

42. Concrete first experiments

The following experiments give high information value without committing to a full platform.

Experiment A - Static manifest inference

Parse the current Tiny-IDP examples, infer capability/outcome/effect sets, and compare them to declared `LambdaSpec` values. Measure false positives, dynamic constructs, and diagnostic quality.

Experiment B - One durable form transform

Compile a linear signup function with one `await durable.form` into the existing explicit handler/continuation API. Verify canonical program equality and run existing workflow tests against the generated form.

Experiment C - Carry leak checker

Introduce nominal types for secret handles, evidence, capability functions, and public values in the internal analyzer. Test rejection of secret, Promise, function, cyclic object, and oversized public carry.

Experiment D - Task quiescence audit

Instrument Promise creation and native capability settlement using available Goja hooks and source transforms. Compare the observed job graph with the current pending-capability counter. This determines whether a VM job-queue patch is necessary.

Experiment E - Fuel prototype

Patch one Goja execution loop to count instructions, run Goja's tests and representative Tiny-IDP handlers, and estimate overhead. Do not merge project-specific semantics into the patch.

Experiment F - Dialect definition prototype

Describe `present.form`, `challenge.email-code`, and four commit effects declaratively. Generate bindings, verifier stubs, documentation, and fake test operations. This tests whether the proposed extension model removes more code than it adds.

43. Final recommendations

43.1 Preserve the current authority split

JavaScript should remain policy and orchestration code inside a native identity kernel. Compiler improvements should make the boundary sharper, not move protocol, secret, persistence, or transaction authority into the language.

43.2 Make semantics independent of Goja early

Use Goja as the parser and execution backend, but define a stable MIR and activation certificate before introducing many language features. Durable records must bind to semantic handler and schema identity, not Goja bytecode or VM frames.

43.3 Treat durable `await` as the flagship feature

It offers the largest usability improvement while strengthening, rather than weakening, the explicit continuation architecture. The compiler should expose the generated state machine and carry schemas so the sugar remains auditable.

43.4 Build an effect and region system before custom syntax

Capability inference, outcome closure, secret lifetime, continuation safety, and structured concurrency provide more value than novel syntax. Standard JavaScript with strong diagnostics is preferable to an elegant syntax without rigorous semantics.

43.5 Fork Goja only for measured missing guarantees

Instruction fuel, allocation accounting, job-queue quiescence, provenance, and tracing are credible reasons. Tiny-IDP-specific opcodes, serialized frames, or a broad rewrite are not.

43.6 Verify the small core

Formalize MIR and its verifier; prove critical lowering properties; differentially test every backend. Avoid claims that the whole ECMAScript implementation or identity provider is verified.

43.7 Design extensions as worlds and dialects

Worlds make authority reviewable. Dialects make semantics reusable. Declarative definitions and generic operation interfaces keep the platform extensible without giving every project arbitrary access to compiler internals.

Appendix A - Proposed core interfaces Contents

A.1 World manifest

```

TEXT
WorldID      string
WorldVersion semver
LanguageProfile ProfileID
Dialects     []DialectRequirement
Imports      []CapabilityImport
Exports      []EntryPoint
Limits       ResourcePolicy
Trust        TrustPolicy

```

A.2 Dialect operation

```

TEXT
OperationDef {
  ID          OperationID
  Version      semver
  OperandTypes []TypeExpr
  ResultTypes  []TypeExpr
  Effects      EffectRow
  Traits       []TraitID
  Regions      RegionRule
  Verifier     DeclarativeRuleSet | TrustedVerifierID
  Lowerings    map[BackendID] LoweringID
  Redaction    RedactionPolicy
  Compatibility CompatibilityPolicy
}

```

A.3 MIR function

```

TEXT
Function {
  ID          StableFunctionID
  Inputs      []TypedValue
  Output      Type
  Effects      EffectRow
  Outcomes    VariantSet
  Blocks      []Block
  ResourceBudget Budget
  SourceMap    ProvenanceMap
}

```

A.4 Suspension terminator

```

TEXT
Suspend {
  Operation      OperationID
  Arguments      []ValueID
  ResumeFunction StableFunctionID
  CarrySchema    SchemaID
  CarryValues    []ValueID
  ExpiryPolicy   ExpiryPolicyID
  BindingPolicy  BindingPolicyID
}

```

Appendix B - Static check catalogue Contents

Check	Stage	Result
Disallowed syntax/profile feature	Parse/profile	Reject
Dynamic capability property	Type/effect	Reject or conservatively widen by world policy
Undeclared capability/effect	Link	Reject
Missing outcome branch	Type/control	Reject or warning by profile
Secret in output/carry/log	Region/information flow	Reject
Invocation resource after durable wait	Liveness/region	Reject
Detached Promise	Task analysis	Reject
Unjournalled nondeterminism before suspension	Determinism	Reject
Mutable top-level state	World policy	Reject or mark worker-stateful
Oversized statically known carry	Resource analysis	Reject
Potential dynamic budget excess	Runtime policy	Meter and fail-stop
Continuation schema incompatibility	Upgrade analysis	Require migration/retention/revocation
Illegal effect ordering	Dialect verifier	Reject
Unknown dialect/op version	Link/activation	Reject
Missing backend lowering	Link	Reject
Certificate/hash mismatch	Activation	Reject
Pending jobs after invocation	Runtime quiescence	Discard worker
Runtime interruption uncertainty	Runtime lifecycle	Discard worker

Appendix C - Example activation certificate [Contents](#)

JSON

```
{
  "format": "mjs-certificate/v1",
  "sourceHash": "sha256:...",
  "frontend": "mjs-js/0.4.0",
  "profile": "assurance-js/1.3",
  "world": "tinyidp.signup/4.0.0",
  "dialects": {
    "core.control": "1.1.0",
    "core.schema": "2.0.0",
    "tinyidp.workflow": "3.2.0",
    "tinyidp.challenge": "2.1.0",
    "tinyidp.commit": "4.0.0"
  },
  "pipelineHash": "sha256:...",
  "mirHash": "sha256:...",
  "imports": ["invitation.inspect@1"],
  "effects": [
    "identity.create@2",
    "credential.bind-password@2",
    "invitation.consume@1",
    "session.establish@1"
  ],
  "handlers": [
    "signup.entry@4",
    "signup.after_form@4",
    "signup.after_email@4"
  ],
  "continuations": {
    "signup.after_form@4": "sha256:...",
    "signup.after_email@4": "sha256:..."
  },
  "verified": {
    "wellTyped": true,
    "regionsSafe": true,
    "worldImportsSatisfied": true,
    "effectSequencesValid": true,
    "continuationCarryBounded": true
  },
  "signature": "ed25519:..."
}
```

Appendix D - Comparison with adjacent systems Contents

System	Relevant lesson	What not to copy blindly
HardenedJS/SES	Frozen primordials, compartments, explicit endowments, deterministic subset	A shim alone does not provide CPU/memory isolation or Tiny-IDP semantics
Starlark	Small deterministic hermetic language, finite execution, frozen module values	Python-like syntax and build-specific semantics are not required
CEL	Checked canonical AST, explicit typed environment, control-plane compilation	Expression-only and non-Turing-complete model cannot express full workflows
Cedar	Purpose-built analyzable policy language, verified validator, differential testing	Authorization semantics do not generalize directly to durable identity workflows
WIT Component Model	Worlds, typed imports/exports, resources, package versions	WebAssembly need not be the first execution backend
MLIR/IRDL	Dialects, interfaces, declarative verifiers, extensible IR and passes	General compiler complexity should be reduced to the project's needs
Koka	Effect rows and handlers make control/effects compositional	A new strongly typed functional source language would sacrifice JS familiarity
Restate/Temporal	Durable normal-code style through journaling/replay and explicit durable operations	Replaying arbitrary identity code without strict effect and continuation contracts is insufficient
QuickJS	Memory limits, interrupt hooks, compact compiler/VM implementation	Version-specific unvalidated bytecode is not a durable or untrusted artifact format
Wasmtime	Fuel, epoch interruption, resource limiting	WebAssembly's instruction model does not directly solve host semantic correctness
eBPF verifier	Admission verification plus bounded execution	Full JavaScript cannot meet eBPF's restricted control/data model without a profile
JS CERT/KJS	Formal executable semantics and conformance testing are possible	Full modern ECMAScript formalization is too broad as the first verification target

Appendix E - Glossary Contents

Activation artifact - Immutable compiled package plus canonical semantic program, source maps, fingerprints, and certificate.

Capability - An explicit reference or imported operation that confers bounded authority.

Continuation carry - The minimal schema-checked values and native references persisted for a future resume handler.

Defunctionalization - Replacing higher-order continuations or closures with a finite data representation and an interpreter that dispatches on stable labels.

Dialect - A versioned package of MIR types, operations, effects, verifiers, and lowerings for one domain.

Durable await - Source syntax that compiles to an explicit persistence boundary, continuation record, and fresh resume invocation.

Effect row - The finite set of abstract operations a function may perform.

Fuel - Deterministic runtime budget charged as VM or MIR operations execute.

MIR - The stable Malleable JS semantic intermediate representation.

Profile - A named, versioned subset and policy for accepted JavaScript syntax and semantics.

Region - A lifetime domain such as static, invocation, secret, transaction, or continuation.

Semantic identity - Hash and stable IDs derived from canonical MIR and its versioned world/dialect context.

World - A project-level contract selecting allowed dialects, imported capabilities, exported entry points, limits, and trust policy.

References

Contents

- [1] Go-Go-Golems, “Lambda-first Tiny-IDP JavaScript API with explicit browser continuations,” pinned Tiny-IDP design document, 2026. <https://github.com/go-go-golems/tiny-idp/blob/d164ae59408bdd8bc21516274b446339b1761b1e/ttmp/2026/07/10/TINYIDP-GOJA-001-go-go-goja-identity-microkernel-scripting-layer/design-doc/03-lambda-first-tiny-idp-javascript-api-with-explicit-browser-continuations.md>
- [2] Go-Go-Golems, Tiny-IDP pkg/idpscript compiler and runtime factory, pinned commit. <https://github.com/go-go-golems/tiny-idp/tree/d164ae59408bdd8bc21516274b446339b1761b1e/pkg/idpscript>
- [3] Ecma International TC39, “ECMAScript 2025 Language Specification,” especially Hosts, Execution Contexts, and Jobs. <https://tc39.es/ecma262/2025/>
- [4] TC39, “Test262: Official ECMAScript Conformance Test Suite.” <https://github.com/tc39/test262>
- [5] Endo, “Endo and HardenedJS (SES) Programming Guide.” <https://docs.endojs.org/documents/guide.html>
- [6] LLVM Project, “Defining Dialects” and “IRDL Dialect,” MLIR documentation. <https://mlir.llvm.org/docs/DefiningDialects/> and <https://mlir.llvm.org/docs/Dialects/IRDL/>
- [7] Cedar Policy, “Policy validation,” and Cedar formal specification repositories. <https://docs.cedarpolicy.com/policies/validation.html> and <https://github.com/cedar-policy/cedar-spec>
- [8] Google, “Common Expression Language specification and binary representation.” <https://github.com/google/cel-spec>
- [9] Dop251, Goja package documentation and pinned source. <https://pkg.go.dev/github.com/dop251/goja> and <https://github.com/dop251/goja/tree/af2ceb9156d7feaff65273b8bfde778077fb4b7e>
- [10] JSCert project, “Certified JavaScript.” <https://jscert.org/>
- [11] D. Park, A. Stefanescu, and G. Roşu, “KJS: A Complete Formal Semantics of JavaScript,” PLDI 2015. <https://fsl.cs.illinois.edu/publications/park-stefanescu-rosu-2015-pldi.html>
- [12] Go-Go-Golems, Tiny-IDP request-time invocation implementation. <https://github.com/go-go-golems/tiny-idp/blob/d164ae59408bdd8bc21516274b446339b1761b1e/pkg/idpscript/invoke.go>
- [13] Go-Go-Golems, Tiny-IDP invocation capability implementation. <https://github.com/go-go-golems/tiny-idp/blob/d164ae59408bdd8bc21516274b446339b1761b1e/pkg/idpscript/capabilities.go>
- [14] Go-Go-Golems, Tiny-IDP durable continuation types. <https://github.com/go-go-golems/tiny-idp/blob/d164ae59408bdd8bc21516274b446339b1761b1e/pkg/idpcontinuation/types.go>
- [15] Bazel Project, “Starlark Language” and specification. <https://github.com/bazelbuild/starlark> and <https://github.com/bazelbuild/starlark/blob/master/spec.md>
- [16] Go-Go-Golems, Tiny-IDP LambdaSpec. <https://github.com/go-go-golems/tiny-idp/blob/d164ae59408bdd8bc21516274b446339b1761b1e/pkg/idpprogram/lambda.go>
- [17] Koka Project, “Effect Handlers,” Koka language book. <https://koka-lang.github.io/koka/doc/book.html>
- [18] TC39, “ShadowRealm Proposal.” <https://github.com/tc39/proposal-shadowrealm>
- [19] Restate, “Durable Execution,” “Concurrent Tasks,” and external-event documentation. <https://docs.restate.dev/foundations/key-concepts> and <https://docs.restate.dev/develop/go/concurrent-tasks>
- [20] Go-Go-Golems, Tiny-IDP Goja native module and runtime-local nominal handle maps. <https://github.com/go-go-golems/tiny-idp/blob/d164ae59408bdd8bc21516274b446339b1761b1e/internal/gojamodules/tinyidp/module.go>
- [21] Go-Go-Golems, Tiny-IDP closed outcomes and native effect plans. <https://github.com/go-go-golems/tiny-idp/blob/d164ae59408bdd8bc21516274b446339b1761b1e/pkg/idpprogram/outcomes.go>
- [22] Bytecode Alliance, Wasmtime resource limiting, fuel, and epoch interruption documentation. <https://docs.wasmtime.dev/api/wasmtime/struct.Config.html>
- [23] Linux Kernel documentation, “eBPF verifier.” <https://docs.kernel.org/bpf/verifier.html>
- [24] Fabrice Bellard and Charlie Gordon, “QuickJS JavaScript Engine,” C API and internals. <https://bellard.org/quickjs/quickjs.html>
- [25] Go-Go-Golems, Tiny-IDP runtime worker pool and fail-stop replacement. <https://github.com/go-go-golems/tiny-idp/blob/d164ae59408bdd8bc21516274b446339b1761b1e/pkg/idpscript/pool.go>
- [26] Dop251, Goja VM execution-context suspension and resumption, pinned source. <https://github.com/dop251/goja/blob/af2ceb9156d7feaff65273b8bfde778077fb4b7e/vm.go>

- [27] LLVM Project, “Interfaces,” MLIR documentation. <https://mlir.llvm.org/docs/Interfaces/>
- [28] Bytecode Alliance, “WIT Worlds” and WIT reference, WebAssembly Component Model. <https://component-model.bytecodealliance.org/design/worlds.html> and <https://component-model.bytecodealliance.org/design/wit.html>
- [29] Dop251, Goja AST package warning, pinned source. <https://github.com/dop251/goja/blob/af2ceb9156d7feaff65273b8bfde778077fb4b7e/ast/node.go>